

10_final_inverse_design_pipeline

April 28, 2026

1 10 - Final Inverse Design Pipeline

This notebook is the clean final inverse-design notebook for the Boom Challenge.

It follows the strongest logic from 06b_inverse_design_robust_refinement:

1. Use the final forward model from notebook 09 as the main surrogate simulator.
2. Generate candidate impact scenarios inside the legal bounds from `constraints.json`.
3. Reuse promising candidate regions from previous inverse-design exploration when available.
4. Add a structured local grid around the promising region.
5. Train a small ensemble for the two constraint targets: P80 and R95.
6. Filter candidates using final-model feasibility and ensemble feasibility.
7. Add a boundary penalty to avoid designs too close to the allowed input limits.
8. Select 20 diverse robust designs.

Final output:

```
outputs/submissions/design_submission.csv
outputs/submissions/design_submission_final_robust.csv
reports/final_inverse_design_selected_diagnostics.csv
reports/final_inverse_design_candidate_pool.csv
```

```
[1]: from pathlib import Path
import json
import itertools
import warnings
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import ExtraTreesRegressor

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)
```

```

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
↳cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"

for path in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR]:
    path.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Forward dir:", FORWARD_DIR)
print("Inverse dir:", INVERSE_DIR)
print("Submissions dir:", SUBMISSIONS_DIR)
print("Models dir:", MODELS_DIR)
print("Reports dir:", REPORTS_DIR)

```

```

Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Forward dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/forward_prediction
Inverse dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/inverse_design
Submissions dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions
Models dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models
Reports dir: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports

```

1.1 1. Load data and constraints

The inverse design task requires 20 impact scenarios with:

P80 <= P80 <= 101

R95 <= 175

The generated input parameters must also respect the legal bounds in `constraints.json`.

```

[2]: input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor",
]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize",
]

```

```

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]

constraints_path = INVERSE_DIR / "constraints.json"
if not constraints_path.exists():
    constraints_path = PROJECT_ROOT / "constraints.json"

with open(constraints_path, "r") as f:
    constraints_data = json.load(f)

output_constraints = constraints_data["constraints"]
input_bounds = constraints_data["input_bounds"]

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]
p80_center = (p80_min + p80_max) / 2

print("Raw train:", raw_train.shape)
print("Targets:", y.shape)
print("Constraints path:", constraints_path)
print("Output constraints:")
display(pd.DataFrame([output_constraints]))
print("Input bounds:")
display(pd.DataFrame(input_bounds).T)

```

Raw train: (2930, 8)

Targets: (2930, 6)

Constraints path: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/data/raw/inverse_design/constraints.json

Output constraints:

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

Input bounds:

	min	max
energy	0.500000	5.000000
angle_rad	0.261799	1.570796
coupling	0.200000	1.700000
strength	0.400000	4.200000
porosity	0.000000	0.330000
gravity	1.020000	10.470000
atmosphere	0.000000	1.000000
shape_factor	0.700000	1.500000

1.2 2. Final forward model class definitions

The saved joblib model from notebook 09 needs these class definitions available before loading. This section must stay aligned with 09_final_forward_prediction_pipeline.ipynb.

```
[3]: class PhysicsFeatureEngineer(BaseEstimator, TransformerMixin):
    def __init__(self, use_advanced: bool = False):
        self.use_advanced = use_advanced

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = pd.DataFrame(X).copy()
        X = X[input_cols].copy()
        eps = 1e-9

        # 1. Energy transfer features
        X["effective_energy"] = X["energy"] * X["coupling"]
        X["log_energy"] = np.log1p(X["energy"])
        X["log_effective_energy"] = np.log1p(X["effective_energy"])

        # 2. Angle decomposition
        X["sin_angle"] = np.sin(X["angle_rad"])
        X["cos_angle"] = np.cos(X["angle_rad"])
        X["tan_angle"] = np.tan(X["angle_rad"])
        X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
        X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
        X["vertical_horizontal_ratio"] = X["vertical_energy"] / X["horizontal_energy"]
        X["horizontal_energy"] += eps

        # 3. Material / fragmentation proxies
        X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
        X["effective_energy_per_strength"] = X["effective_energy"] / X["strength"]
        X["strength"] += eps
        X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
        X["fragmentation_index"] = X["effective_energy"] * X["porosity"] / X["strength"]
        X["strength"] += eps
        X["coupling_porosity"] = X["coupling"] * X["porosity"]
        X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
        X["porosity_strength"] = X["porosity"] * X["strength"]

        # 4. Gravity and range proxies
        X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
        X["effective_energy_per_gravity"] = X["effective_energy"] / X["gravity"]
        X["gravity"] += eps
        X["horizontal_energy_per_gravity"] = X["horizontal_energy"] / X["gravity"]
        X["gravity"] += eps
```

```

X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"] +
↪ eps)

# 5. Atmosphere and drag proxies
X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *
↪ X["effective_energy"]
X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

# 6. Pi-like scaling proxies
X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] +
↪ eps)
X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"] +
↪ eps)
X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] *
↪ X["coupling"] + eps)

# 7. Regime indicators from EDA
X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
X["strength_regime"] = (X["strength"] > 2.6).astype(int)
X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
X["regime_combo"] = (
    X["porosity_regime"] * 8
    + X["strength_regime"] * 4
    + X["angle_regime"] * 2
    + X["atm_regime"]
)

# 8. Cross-regime proxies
X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
↪ sqrt(X["gravity"]) + eps)
X["fragility"] = X["porosity"] / (X["strength"] + eps)
X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) /
↪ (X["gravity"] * X["strength"] + eps)
X["energy_sin_angle"] = X["energy"] * X["sin_angle"]
X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]
X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] /
↪ (X["energy"] + eps)

# 9. Validated advanced features, used only for fragmentation targets
if self.use_advanced:

```

```

        X["froude_proxy"] = np.sqrt(X["effective_energy"] / (X["gravity"] +
↪eps))
        X["stress_ratio_compact"] = X["effective_energy"] /
↪(X["material_resistance_index"] + eps)
        X["sqrt_effective_energy_per_strength"] = np.
↪sqrt(X["effective_energy_per_strength"].clip(lower=0))
        X["sqrt_effective_energy_per_gravity"] = np.
↪sqrt(X["effective_energy_per_gravity"].clip(lower=0))

    return X

class ColumnSelector(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = list(columns)

    def fit(self, X, y=None):
        missing = [c for c in self.columns if c not in X.columns]
        if missing:
            raise ValueError(f"Missing columns: {missing}")
        return self

    def transform(self, X):
        return X[self.columns].copy()

def build_extratrees(random_state=42):
    return ExtraTreesRegressor(
        n_estimators=800,
        max_features="sqrt",
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )

def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()
    if target in ["fines_frac", "oversize_frac"]:
        return np.clip(preds, 0, 1)
    return np.clip(preds, 0, None)

# Feature lists used by the final model
raw_features = input_cols.copy()

fragmentation_features_v1 = raw_features + [

```

```

    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",␣
    ↪ "regime_combo",
]

fragmentation_features_v2 = fragmentation_features_v1 + [
    "stress_ratio_compact",
    "sqrt_effective_energy_per_strength",
]

distance_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",␣
    ↪ "atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",␣
    ↪ "regime_combo",
]

final_target_config = {
    "P80": {"model": "ExtraTrees", "feature_version": "v2", "features":␣
    ↪ fragmentation_features_v2},
    "fines_frac": {"model": "ExtraTrees", "feature_version": "v2", "features":␣
    ↪ fragmentation_features_v2},
    "oversize_frac": {"model": "ExtraTrees", "feature_version": "v2",␣
    ↪ "features": fragmentation_features_v2},
    "R95": {"model": "ExtraTrees", "feature_version": "v1", "features":␣
    ↪ distance_features_v1},
    "R50_fines": {"model": "ExtraTrees", "feature_version": "v1", "features":␣
    ↪ distance_features_v1},
    "R50_oversize": {"model": "ExtraTrees", "feature_version": "v1", "features":
    ↪ distance_features_v1},
}

```

```

def build_target_pipeline(target, random_state=42):
    cfg = final_target_config[target]
    use_advanced = cfg["feature_version"] == "v2"
    return Pipeline(steps=[
        ("features", PhysicsFeatureEngineer(use_advanced=use_advanced)),
        ("select", ColumnSelector(cfg["features"])),
        ("model", build_extratrees(random_state=random_state)),
    ])

class FinalForwardPipeline:
    def __init__(self, target_config, random_state=42):
        self.target_config = target_config
        self.random_state = random_state
        self.pipelines_ = {}

    def fit(self, X_raw, y_df):
        for i, target in enumerate(target_cols):
            pipe = build_target_pipeline(target, random_state=self.random_state,
            ↪ i)
            pipe.fit(X_raw, y_df[target])
            self.pipelines_[target] = pipe
        return self

    def predict(self, X_raw):
        preds = pd.DataFrame(index=X_raw.index)
        for target in target_cols:
            pred = self.pipelines_[target].predict(X_raw)
            preds[target] = clip_predictions(pred, target)
        return preds[target_cols]

    def describe(self):
        rows = []
        for target, pipe in self.pipelines_.items():
            cfg = self.target_config[target]
            rows.append({
                "target": target,
                "model": cfg["model"],
                "feature_version": cfg["feature_version"],
                "n_features": len(cfg["features"]),
                "pipeline_model": type(pipe.named_steps["model"]).__name__,
            })
        return pd.DataFrame(rows)

```

1.3 3. Load the final forward model from notebook 09

If the saved model is missing, this notebook rebuilds the same final model from the training data.

```
[4]: final_model_path = MODELS_DIR / "final_forward_pipeline.joblib"

try:
    final_forward_model = joblib.load(final_model_path)
    print("Loaded final forward model from:", final_model_path)
except Exception as e:
    print("Could not load final_forward_pipeline.joblib. Rebuilding from_
    ↪training data.")
    print("Reason:", e)
    final_forward_model =_
    ↪FinalForwardPipeline(target_config=final_target_config, random_state=42)
    final_forward_model.fit(raw_train, y)

print("Final forward model description:")
display(final_forward_model.describe())
```

Loaded final forward model from: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_forward_pipeline.joblib
 Final forward model description:

	target	model	feature_version	n_features	pipeline_model
0	P80	ExtraTrees	v2	36	ExtraTreesRegressor
1	fines_frac	ExtraTrees	v2	36	ExtraTreesRegressor
2	oversize_frac	ExtraTrees	v2	36	ExtraTreesRegressor
3	R95	ExtraTrees	v1	33	ExtraTreesRegressor
4	R50_fines	ExtraTrees	v1	33	ExtraTreesRegressor
5	R50_oversize	ExtraTrees	v1	33	ExtraTreesRegressor

1.4 4. Constraint ensemble for P80 and R95

The final model is the main simulator. The ensemble is used only to estimate stability around the inverse-design constraints. A candidate is safer when:

- the final model predicts it as feasible;
- the ensemble mean predicts it as feasible;
- the ensemble disagreement is small;
- the candidate is not too close to the input bounds.

```
[5]: # Optional model families for uncertainty checking.
try:
    from catboost import CatBoostRegressor

    def build_catboost(random_state=42):
        return CatBoostRegressor(
            iterations=1200,
            learning_rate=0.03,
            depth=6,
            l2_leaf_reg=5.0,
            loss_function="RMSE",
```

```

        random_seed=random_state,
        verbose=False,
    )

    CATBOOST_AVAILABLE = True
except Exception as e:
    print("CatBoost unavailable:", e)
    CATBOOST_AVAILABLE = False

try:
    from xgboost import XGBRegressor

    def build_xgb(random_state=42):
        return XGBRegressor(
            n_estimators=900,
            learning_rate=0.03,
            max_depth=4,
            min_child_weight=2,
            subsample=0.85,
            colsample_bytree=0.85,
            reg_alpha=0.05,
            reg_lambda=2.0,
            objective="reg:squarederror",
            random_state=random_state,
            n_jobs=-1,
        )

    XGB_AVAILABLE = True
except Exception as e:
    print("XGBoost unavailable:", e)
    XGB_AVAILABLE = False

try:
    from lightgbm import LGBMRegressor

    def build_lgbm(random_state=42):
        return LGBMRegressor(
            n_estimators=1200,
            learning_rate=0.025,
            num_leaves=31,
            min_child_samples=20,
            subsample=0.85,
            colsample_bytree=0.85,
            reg_alpha=0.05,
            reg_lambda=2.0,
            random_state=random_state,
            n_jobs=-1,

```

```

        verbosity=-1,
    )

    LGBM_AVAILABLE = True
except Exception as e:
    print("LightGBM unavailable:", e)
    LGBM_AVAILABLE = False

print("CatBoost:", CATBOOST_AVAILABLE, "XGBoost:", XGB_AVAILABLE, "LightGBM:", LGBM_AVAILABLE)

```

CatBoost: True XGBoost: True LightGBM: True

```

[6]: # Full feature matrices for ensemble training.
X_base_train = PhysicsFeatureEngineer(use_advanced=False).
    fit_transform(raw_train)
X_v2_train = PhysicsFeatureEngineer(use_advanced=True).fit_transform(raw_train)

# Alternative feature sets for ensemble uncertainty only.
extended_plus_regimes_features = raw_features + [
    "effective_energy", "log_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "tan_angle",
    "horizontal_energy", "vertical_energy", "vertical_horizontal_ratio",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    "atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_strength_proxy", "pi_atmosphere_proxy",
    "scaled_energy", "fragility", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "coupling_per_atm_clipped", "log_coupling_per_atm", "retention_factor",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    "regime_combo",
]

extended_plus_regimes_features_v2 = extended_plus_regimes_features + [
    "froude_proxy",
    "stress_ratio_compact",
    "sqrt_effective_energy_per_strength",
    "sqrt_effective_energy_per_gravity",
]

constraint_targets = ["P80", "R95"]

```

```

ensemble_specs = [
    ("ExtraTrees_final_style", build_extratrees, "final_target"),
    ("ExtraTrees_extended_v2", build_extratrees, "extended_v2"),
]

if CATBOOST_AVAILABLE:
    ensemble_specs.append(("CatBoost_final_style", build_catboost,
        ↪ "final_target"))
    ensemble_specs.append(("CatBoost_extended_v2", build_catboost,
        ↪ "extended_v2"))
if XGB_AVAILABLE:
    ensemble_specs.append(("XGBoost_final_style", build_xgb, "final_target"))
    ensemble_specs.append(("XGBoost_extended_v2", build_xgb, "extended_v2"))
if LGBM_AVAILABLE:
    ensemble_specs.append(("LightGBM_final_style", build_lgbm, "final_target"))
    ensemble_specs.append(("LightGBM_extended_v2", build_lgbm, "extended_v2"))

print("Ensemble specs:")
for name, _, fset in ensemble_specs:
    print("-", name, "|", fset)

def get_ensemble_matrix_and_features(feature_set_name, target):
    if feature_set_name == "final_target":
        cfg = final_target_config[target]
        use_advanced = cfg["feature_version"] == "v2"
        X_matrix = X_v2_train if use_advanced else X_base_train
        return X_matrix, cfg["features"]
    if feature_set_name == "extended_v2":
        return X_v2_train, extended_plus_regimes_features_v2
    raise ValueError(feature_set_name)

ensemble_models = {target: [] for target in constraint_targets}

for target in constraint_targets:
    print(f"Training ensemble models for {target}...")
    for i, (name, builder, feature_set_name) in enumerate(ensemble_specs):
        X_matrix, features = get_ensemble_matrix_and_features(feature_set_name,
            ↪ target)
        model = builder(random_state=1000 + i)
        model.fit(X_matrix[features], y[target])
        ensemble_models[target].append({
            "name": name,
            "model": model,
            "feature_set_name": feature_set_name,
            "features": features,
        })

```

```
print(" trained", name)
```

Ensemble specs:

```
- ExtraTrees_final_style | final_target
- ExtraTrees_extended_v2 | extended_v2
- CatBoost_final_style | final_target
- CatBoost_extended_v2 | extended_v2
- XGBoost_final_style | final_target
- XGBoost_extended_v2 | extended_v2
- LightGBM_final_style | final_target
- LightGBM_extended_v2 | extended_v2
```

Training ensemble models for P80...

```
trained ExtraTrees_final_style
trained ExtraTrees_extended_v2
trained CatBoost_final_style
trained CatBoost_extended_v2
trained XGBoost_final_style
trained XGBoost_extended_v2
trained LightGBM_final_style
trained LightGBM_extended_v2
```

Training ensemble models for R95...

```
trained ExtraTrees_final_style
trained ExtraTrees_extended_v2
trained CatBoost_final_style
trained CatBoost_extended_v2
trained XGBoost_final_style
trained XGBoost_extended_v2
trained LightGBM_final_style
trained LightGBM_extended_v2
```

1.5 5. Candidate generation

This uses the robust logic that worked best:

- reuse the best regions discovered previously when available;
- create a local grid inside the promising region;
- add near-feasible training anchors;
- optionally add a small global Latin Hypercube sample for exploration.

```
[7]: RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)

def bounds_arrays(input_bounds, input_cols):
    lower = np.array([input_bounds[c]["min"] for c in input_cols], dtype=float)
    upper = np.array([input_bounds[c]["max"] for c in input_cols], dtype=float)
    return lower, upper
```

```

lower_bounds, upper_bounds = bounds_arrays(input_bounds, input_cols)

def sample_latin_hypercube(n_samples: int, random_state: int = 42) -> pd.
    DataFrame:
    try:
        from scipy.stats import qmc
        sampler = qmc.LatinHypercube(d=len(input_cols), seed=random_state)
        sample = sampler.random(n_samples)
        values = qmc.scale(sample, lower_bounds, upper_bounds)
    except Exception as e:
        print("Latin Hypercube unavailable. Falling back to uniform sampling.")
        print("Reason:", e)
        values = rng.uniform(lower_bounds, upper_bounds, size=(n_samples,
    len(input_cols)))
    return pd.DataFrame(values, columns=input_cols)

def feasibility_mask(df_targets):
    return (df_targets["P80"].between(p80_min, p80_max)) & (df_targets["R95"]
    <= r95_max)

def near_feasible_mask(df_targets):
    return (df_targets["P80"].between(80, 120)) & (df_targets["R95"] <= 250)

true_feasible_train = feasibility_mask(y)
near_feasible_train = near_feasible_mask(y)

print("True feasible train rows:", int(true_feasible_train.sum()))
print("Near feasible train rows:", int(near_feasible_train.sum()))

```

True feasible train rows: 35

Near feasible train rows: 372

```

[8]: # Reuse strong seeds from 06/06b when available. If not, generate a seed pool.
seed_paths = [
    REPORTS_DIR / "inverse_design_candidate_pool.csv",
    REPORTS_DIR / "final_inverse_design_candidate_pool.csv",
]

seed_candidates = None
for path in seed_paths:
    if path.exists():
        print("Loading seed candidates from:", path)
        tmp = pd.read_csv(path)
        if all(c in tmp.columns for c in input_cols):

```

```

        seed_candidates = tmp[input_cols].drop_duplicates().
↪reset_index(drop=True)
        break

if seed_candidates is None:
    print("No previous seed pool found. Creating Latin Hypercube seed pool.")
    seed_candidates = sample_latin_hypercube(120_000, random_state=RANDOM_STATE)

# Keep at most the first 25k seed candidates if the previous report is large.
seed_candidates = seed_candidates.head(25_000).copy()
print("Seed candidates:", seed_candidates.shape)
display(seed_candidates.head())

```

Loading seed candidates from: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports/inverse_design_candidate_pool.csv
Seed candidates: (20000, 8)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	
↪shape_factor								
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	↪
↪1.321979								
1	3.801488	0.567512	0.757391	1.385881	0.294003	10.179310	0.839499	↪
↪0.832437								
2	3.217315	0.742617	1.333629	1.811721	0.306086	10.455045	0.664532	↪
↪0.916205								
3	3.223864	0.625563	0.864345	1.346453	0.330000	9.989450	0.616066	↪
↪1.024933								
4	3.857398	0.636890	0.847146	1.632682	0.330000	10.030104	0.822272	↪
↪1.166845								

```

[9]: def create_local_intervals_from_seed(seed_df, input_cols, input_bounds, q_low=0.
↪05, q_high=0.95, padding_ratio=0.02):
    intervals = {}
    for col in input_cols:
        low_allowed = input_bounds[col]["min"]
        high_allowed = input_bounds[col]["max"]
        width_allowed = high_allowed - low_allowed

        local_low = seed_df[col].quantile(q_low) - padding_ratio * width_allowed
        local_high = seed_df[col].quantile(q_high) + padding_ratio *
↪width_allowed

        local_low = max(low_allowed, local_low)
        local_high = min(high_allowed, local_high)
        intervals[col] = (float(local_low), float(local_high))
    return intervals

```

```

local_intervals = create_local_intervals_from_seed(
    seed_candidates,
    input_cols=input_cols,
    input_bounds=input_bounds,
    q_low=0.05,
    q_high=0.95,
    padding_ratio=0.02,
)

local_intervals_df = pd.DataFrame(local_intervals, index=["local_min",
↪ "local_max"]).T
display(local_intervals_df)

```

	local_min	local_max
energy	2.693512	4.585454
angle_rad	0.501950	0.885743
coupling	0.536660	1.573947
strength	0.955697	2.506637
porosity	0.251777	0.330000
gravity	8.922212	10.470000
atmosphere	0.394378	0.864865
shape_factor	0.759789	1.333853

```

[10]: def make_local_grid(local_intervals, points_per_feature=5):
    grid_values = {
        col: np.linspace(low, high, points_per_feature)
        for col, (low, high) in local_intervals.items()
    }
    grid = pd.DataFrame(
        itertools.product(*grid_values.values()),
        columns=grid_values.keys(),
    )
    return grid

# 5^8 = 390,625 candidates. Reduce to 4 if your local machine is too slow.
POINTS_PER_FEATURE = 5
local_grid_candidates = make_local_grid(local_intervals,
↪ points_per_feature=POINTS_PER_FEATURE)
print("Local grid candidates:", local_grid_candidates.shape)
display(local_grid_candidates.head())

```

Local grid candidates: (390625, 8)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	↪
↪shape_factor								
0	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	↪
↪0.759789								

1	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	↳
	↳0.903305							
2	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	↳
	↳1.046821							
3	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	↳
	↳1.190337							
4	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	↳
	↳1.333853							

```
[11]: # Add anchors from known near-feasible training examples.
anchor_candidates = raw_train.loc[near_feasible_train, input_cols].copy()

# Add a small global sample to avoid overfitting only to the known region.
N_GLOBAL_EXTRA = 50_000
global_extra_candidates = sample_latin_hypercube(N_GLOBAL_EXTRA,
↳random_state=RANDOM_STATE + 10)

combined_candidates = pd.concat(
    [seed_candidates, local_grid_candidates, anchor_candidates,
↳global_extra_candidates],
    ignore_index=True,
)

combined_candidates = combined_candidates.round(10).drop_duplicates().
↳reset_index(drop=True)
print("Combined candidates:", combined_candidates.shape)
display(combined_candidates.head())
```

Combined candidates: (460949, 8)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	↳
	↳shape_factor							
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	↳
	↳1.321979							
1	3.801488	0.567512	0.757391	1.385881	0.294003	10.179310	0.839499	↳
	↳0.832437							
2	3.217315	0.742617	1.333629	1.811721	0.306086	10.455045	0.664532	↳
	↳0.916205							
3	3.223864	0.625563	0.864345	1.346453	0.330000	9.989450	0.616066	↳
	↳1.024933							
4	3.857398	0.636890	0.847146	1.632682	0.330000	10.030104	0.822272	↳
	↳1.166845							

1.6 6. Predict candidates and compute ensemble diagnostics

```
[12]: def predict_constraint_ensemble(candidates_raw: pd.DataFrame) -> pd.DataFrame:
    X_base = PhysicsFeatureEngineer(use_advanced=False).
    ↪fit_transform(candidates_raw)
    X_v2 = PhysicsFeatureEngineer(use_advanced=True).
    ↪fit_transform(candidates_raw)

    out = pd.DataFrame(index=candidates_raw.index)

    for target in constraint_targets:
        model_preds = []
        for info in ensemble_models[target]:
            feature_set_name = info["feature_set_name"]
            if feature_set_name == "final_target":
                cfg = final_target_config[target]
                X_matrix = X_v2 if cfg["feature_version"] == "v2" else X_base
            elif feature_set_name == "extended_v2":
                X_matrix = X_v2
            else:
                raise ValueError(feature_set_name)

            pred = info["model"].predict(X_matrix[info["features"]])
            pred = clip_predictions(pred, target)
            model_preds.append(pred)
            out[f"{target}_{info['name']}"] = pred

        pred_matrix = np.vstack(model_preds).T
        out[f"{target}_ens_mean"] = pred_matrix.mean(axis=1)
        out[f"{target}_ens_std"] = pred_matrix.std(axis=1)
        out[f"{target}_ens_min"] = pred_matrix.min(axis=1)
        out[f"{target}_ens_max"] = pred_matrix.max(axis=1)

    return out

print("Predicting candidates with final forward model...")
final_preds = final_forward_model.predict(combined_candidates)

print("Predicting candidates with constraint ensemble...")
ensemble_preds = predict_constraint_ensemble(combined_candidates)

candidate_results = pd.concat(
    [combined_candidates, final_preds.add_prefix("final_"), ensemble_preds],
    axis=1,
)

print("Candidate results:", candidate_results.shape)
```

```
display(candidate_results.head())
```

Predicting candidates with final forward model...

Predicting candidates with constraint ensemble...

Candidate results: (460949, 38)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	
	shape_factor	final_P80	final_fines_frac	final_oversize_frac	final_R95			
	final_R50_fines	\						
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	
	1.321979	98.262437		0.093945		0.069666	85.375134	
	82.498980							
1	3.801488	0.567512	0.757391	1.385881	0.294003	10.179310	0.839499	
	0.832437	99.013662		0.078758		0.072591	75.110355	
	72.395087							
2	3.217315	0.742617	1.333629	1.811721	0.306086	10.455045	0.664532	
	0.916205	98.207411		0.093629		0.068198	89.707164	
	84.950800							
3	3.223864	0.625563	0.864345	1.346453	0.330000	9.989450	0.616066	
	1.024933	99.997736		0.080051		0.080446	78.432664	
	77.893378							
4	3.857398	0.636890	0.847146	1.632682	0.330000	10.030104	0.822272	
	1.166845	98.291961		0.081599		0.074449	76.223801	
	74.906513							

	final_R50_oversize	P80_ExtraTrees_final_style	P80_ExtraTrees_extended_v2	
	P80_CatBoost_final_style	P80_CatBoost_extended_v2	P80_XGBoost_final_style	\
0	37.092522		98.340135	98.141784
	97.126631		98.394868	98.658714
1	32.060300		98.657517	98.550193
	97.074074		96.990857	97.759766
2	38.936681		98.070397	98.703054
	97.250848		98.460815	97.803284
3	35.095644		99.837044	99.635606
	98.316422		99.413099	97.998428
4	35.083795		98.752906	98.121381
	100.734244		99.171614	99.957825

	P80_XGBoost_extended_v2	P80_LightGBM_final_style	P80_LightGBM_extended_v2	
	P80_ens_mean	P80_ens_std	P80_ens_min	P80_ens_max
	R95_ExtraTrees_final_style	\		
0	98.309532		97.867138	98.183483
	98.127786	0.433718	97.126631	98.658714
1	97.603081		99.616458	98.796519
	98.131058	0.861392	96.990857	99.616458
2	98.162460		97.341758	97.270473
	97.882886	0.523651	97.250848	98.703054

3	97.346436	98.825133	98.295331
↪ 98.708437	0.816016	97.346436	99.837044
4	99.994751	97.088194	98.283392
↪ 99.013038	1.114911	100.734244	75.954197

	R95_ExtraTrees_extended_v2	R95_CatBoost_final_style	
	↪ R95_CatBoost_extended_v2	R95_XGBoost_final_style	R95_XGBoost_extended_v2
	↪ R95_LightGBM_final_style	\	
0	86.627243	81.315724	83.
↪ 435329	93.574913	87.311325	88.
↪ 515601			
1	74.799460	80.464459	86.
↪ 412044	77.671082	79.123413	70.
↪ 914273			
2	89.629305	78.595402	82.
↪ 152915	86.542168	85.603142	83.
↪ 924809			
3	77.428775	84.589988	86.
↪ 722347	82.663376	82.491547	82.
↪ 414974			
4	76.111798	75.386544	74.
↪ 915408	75.962784	79.552071	77.
↪ 377862			

	R95_LightGBM_extended_v2	R95_ens_mean	R95_ens_std	R95_ens_min	R95_ens_max
0	83.563085	86.234743	3.536501	81.315724	93.574913
1	69.365712	76.719524	5.111956	69.365712	86.412044
2	82.816553	84.650561	3.295616	78.595402	89.629305
3	75.491493	81.413162	3.474915	75.491493	86.722347
4	79.267396	76.816007	1.636814	74.915408	79.552071

1.7 7. Robust scoring with boundary penalty

The score rewards:

- P80 close to the center of the allowed band;
- low R95;
- low ensemble uncertainty;
- lower energy;
- zero constraint violation;
- low boundary proximity penalty.

The boundary penalty avoids choosing 20 designs that are all stuck at the maximum allowed values of gravity or porosity.

```
[13]: def constraint_violation(p80, r95, p80_min=p80_min, p80_max=p80_max,
↪ r95_max=r95_max):
    return (
```

```

        np.maximum(p80_min - p80, 0)
        + np.maximum(p80 - p80_max, 0)
        + np.maximum(r95 - r95_max, 0)
    )

candidate_results["final_feasible"] = (
    candidate_results["final_P80"].between(p80_min, p80_max)
    & (candidate_results["final_R95"] <= r95_max)
)

candidate_results["ens_feasible"] = (
    candidate_results["P80_ens_mean"].between(p80_min, p80_max)
    & (candidate_results["R95_ens_mean"] <= r95_max)
)

# Conservative safety margin.
candidate_results["safe_feasible"] = (
    candidate_results["P80_ens_mean"].between(97.0, 100.0)
    & (candidate_results["R95_ens_mean"] <= 160.0)
    & (candidate_results["final_P80"].between(96.0, 101.0))
    & (candidate_results["final_R95"] <= 175.0)
)

candidate_results["ens_constraint_violation"] = constraint_violation(
    candidate_results["P80_ens_mean"],
    candidate_results["R95_ens_mean"],
)

candidate_results["final_constraint_violation"] = constraint_violation(
    candidate_results["final_P80"],
    candidate_results["final_R95"],
)

# Lower is better.
candidate_results["design_score"] = (
    8.0 * np.abs(candidate_results["P80_ens_mean"] - p80_center)
    + 0.04 * candidate_results["R95_ens_mean"]
    + 1.5 * candidate_results["P80_ens_std"]
    + 0.04 * candidate_results["R95_ens_std"]
    + 0.20 * candidate_results["energy"]
    + 15.0 * candidate_results["ens_constraint_violation"]
    + 15.0 * candidate_results["final_constraint_violation"]
)

summary_df = pd.DataFrame([
    "total_candidates": len(candidate_results),
    "final_feasible": int(candidate_results["final_feasible"].sum()),

```

```

    "ensemble_feasible": int(candidate_results["ens_feasible"].sum()),
    "safe_feasible": int(candidate_results["safe_feasible"].sum()),
})

display(summary_df)

```

	total_candidates	final_feasible	ensemble_feasible	safe_feasible
0	460949	37769	35543	19030

```

[14]: def boundary_penalty(df, input_cols, input_bounds, margin_ratio=0.05):
    penalty = np.zeros(len(df))
    details = {}

    for col in input_cols:
        low = input_bounds[col]["min"]
        high = input_bounds[col]["max"]
        width = high - low
        margin = margin_ratio * width

        distance_to_low = df[col] - low
        distance_to_high = high - df[col]

        low_penalty = np.maximum(margin - distance_to_low, 0) / (margin + 1e-12)
        high_penalty = np.maximum(margin - distance_to_high, 0) / (margin +
↪1e-12)
        col_penalty = low_penalty + high_penalty

        penalty += col_penalty
        details[f"boundary_penalty_{col}"] = col_penalty

    details_df = pd.DataFrame(details, index=df.index)
    return penalty, details_df

# Weight 3 was effective in 06b: strong enough to avoid boundary-hugging, not
↪too strong.
boundary_weight = 3.0
penalty, penalty_details = boundary_penalty(
    candidate_results,
    input_cols=input_cols,
    input_bounds=input_bounds,
    margin_ratio=0.05,
)

candidate_results["boundary_penalty"] = penalty
candidate_results = pd.concat([candidate_results, penalty_details], axis=1)

candidate_results["robust_design_score"] = (

```

```

        candidate_results["design_score"]
        + boundary_weight * candidate_results["boundary_penalty"]
    )

candidate_pool = candidate_results[
    candidate_results["final_feasible"] & candidate_results["ens_feasible"]
].copy()

safe_pool = candidate_results[candidate_results["safe_feasible"]].copy()

print("Candidate pool:", candidate_pool.shape)
print("Safe pool:", safe_pool.shape)

display(
    safe_pool.sort_values("robust_design_score").head(10)[
        input_cols + [
            "final_P80", "final_R95", "P80_ens_mean", "P80_ens_std",
            "R95_ens_mean", "R95_ens_std",
            "design_score", "boundary_penalty", "robust_design_score",
        ]
    ]
)

```

Candidate pool: (28580, 54)

Safe pool: (19030, 54)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere
↪ shape_factor	final_P80	final_R95	P80_ens_mean	P80_ens_std	R95_ens_mean		
↪ R95_ens_std \							
538	3.598016	0.797647	1.291533	2.024534	0.304489	9.851536	0.422767
↪	1.303159	98.663679	88.973302	98.530183	0.276672	87.933757	
↪	3.950814						
2348	3.219857	0.653336	0.937745	1.371766	0.268351	10.031512	0.819009
↪	0.775107	98.648438	85.952720	98.527925	0.358066	83.469422	
↪	3.691210						
2218	3.883410	0.550208	0.754591	1.428060	0.294039	9.893283	0.800587
↪	0.798041	99.442042	77.115577	98.561214	0.620543	77.828987	
↪	3.509766						
3062	2.769840	0.856038	0.933836	1.186456	0.266355	9.856101	0.429999
↪	1.214199	97.907049	81.647131	98.456395	0.467913	91.252647	
↪	5.175062						
464	3.491508	0.545414	1.620353	2.249113	0.264882	9.960773	0.715197
↪	0.816463	97.735768	83.307290	98.533873	0.661069	83.855754	
↪	5.006998						
622	2.818390	0.657906	0.841158	1.143469	0.291200	9.963445	0.841676
↪	0.994703	97.931658	80.121540	98.528999	0.965068	81.851145	
↪	2.736207						

339	3.944120	0.566188	0.829191	1.568268	0.293000	9.997341	0.808564
↪	1.187302	98.327340	74.922776	98.592481	0.641508	76.829547	
↪	1.873050						
2192	2.972409	0.772728	1.329016	1.663816	0.306724	9.953774	0.682885
↪	0.850362	98.666910	84.670167	98.562062	0.694935	85.022119	
↪	3.444496						
226972	3.639483	0.789794	0.795982	1.343432	0.251777	10.083053	0.864865
↪	1.046821	98.185973	79.786165	98.534630	0.500376	82.380356	
↪	1.958773						
2003	3.787539	0.728334	1.498127	2.416876	0.292087	9.921431	0.611923
↪	0.918703	98.665610	91.801889	98.551545	0.576920	88.184502	
↪	3.256693						

	design_score	boundary_penalty	robust_design_score
538	5.051461	0.000000	5.051461
2348	4.890897	0.071983	5.106846
2218	5.450756	0.000000	5.450756
3062	5.461785	0.000000	5.461785
464	5.515397	0.000000	5.515397
622	5.626766	0.000000	5.626766
339	5.639042	0.000000	5.639042
2192	5.672049	0.000000	5.672049
226972	5.129068	0.181065	5.672262
2003	5.692893	0.000000	5.692893

1.8 8. Diverse selection of 20 final designs

We select candidates from `safe_pool` when possible. If there are not enough safe candidates, we fall back to the wider feasible pool.

```
[15]: def greedy_diverse_selection(pool: pd.DataFrame, score_col: str, n_select: int,
    ↪    min_distance: float = 0.12) -> pd.DataFrame:
    if len(pool) == 0:
        return pool.copy()

    pool_sorted = pool.sort_values(score_col).reset_index(drop=True)

    scaler = MinMaxScaler()
    X_scaled = scaler.fit_transform(pool_sorted[input_cols])

    selected_indices = []
    for i in range(len(pool_sorted)):
        if len(selected_indices) == 0:
            selected_indices.append(i)
        else:
            distances = np.linalg.norm(X_scaled[i] -
    ↪ X_scaled[selected_indices], axis=1)
```

```

        if distances.min() >= min_distance:
            selected_indices.append(i)

    if len(selected_indices) >= n_select:
        break

    return pool_sorted.iloc[selected_indices].copy()

def select_20(pool, score_col):
    selected = pd.DataFrame()
    for threshold in [0.22, 0.20, 0.18, 0.15, 0.12, 0.10, 0.08, 0.05, 0.0]:
        selected = greedy_diverse_selection(pool, score_col=score_col,
        ↪n_select=20, min_distance=threshold)
        print(score_col, "threshold", threshold, "selected", len(selected))
        if len(selected) >= 20:
            break
    return selected.head(20).reset_index(drop=True)

if len(safe_pool) >= 20:
    selection_source = safe_pool
    source_name = "safe_pool"
elif len(candidate_pool) >= 20:
    selection_source = candidate_pool
    source_name = "candidate_pool"
else:
    selection_source = candidate_results.sort_values([
        "ens_constraint_violation", "final_constraint_violation",
        ↪"robust_design_score"
    ]).head(5000).copy()
    source_name = "lowest_violation_pool"

print("Selection source:", source_name, selection_source.shape)

selected_designs = select_20(selection_source, score_col="robust_design_score")
print("Selected designs:", selected_designs.shape)

display(
    selected_designs[input_cols + [
        "final_P80", "final_R95", "final_fines_frac", "final_oversize_frac",
        "final_R50_fines", "final_R50_oversize",
        "P80_ens_mean", "P80_ens_std", "R95_ens_mean", "R95_ens_std",
        "final_feasible", "ens_feasible", "safe_feasible",
        "design_score", "boundary_penalty", "robust_design_score",
    ]]
)

```

Selection source: safe_pool (19030, 54)
robust_design_score threshold 0.22 selected 20
Selected designs: (20, 54)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	
↪shape_factor	final_P80	final_R95	final_fines_frac	final_oversize_frac				
↪final_R50_fines	\							
0	3.598016	0.797647	1.291533	2.024534	0.304489	9.851536	0.422767	↪
↪	1.303159	98.663679	88.973302		0.093206		0.072531	↪
↪	86.308420							
1	3.219857	0.653336	0.937745	1.371766	0.268351	10.031512	0.819009	↪
↪	0.775107	98.648438	85.952720		0.084552		0.072528	↪
↪	83.312234							
2	3.883410	0.550208	0.754591	1.428060	0.294039	9.893283	0.800587	↪
↪	0.798041	99.442042	77.115577		0.077125		0.074867	↪
↪	73.694526							
3	2.769840	0.856038	0.933836	1.186456	0.266355	9.856101	0.429999	↪
↪	1.214199	97.907049	81.647131		0.086700		0.062362	↪
↪	81.902085							
4	3.491508	0.545414	1.620353	2.249113	0.264882	9.960773	0.715197	↪
↪	0.816463	97.735768	83.307290		0.099837		0.071675	↪
↪	76.578171							
5	2.818390	0.657906	0.841158	1.143469	0.291200	9.963445	0.841676	↪
↪	0.994703	97.931658	80.121540		0.082807		0.064294	↪
↪	78.788268							
6	3.944120	0.566188	0.829191	1.568268	0.293000	9.997341	0.808564	↪
↪	1.187302	98.327340	74.922776		0.081634		0.068449	↪
↪	72.164253							
7	2.972409	0.772728	1.329016	1.663816	0.306724	9.953774	0.682885	↪
↪	0.850362	98.666910	84.670167		0.095811		0.071173	↪
↪	80.211657							
8	3.639483	0.789794	0.795982	1.343432	0.251777	10.083053	0.864865	↪
↪	1.046821	98.185973	79.786165		0.083235		0.070242	↪
↪	77.261703							
9	3.787539	0.728334	1.498127	2.416876	0.292087	9.921431	0.611923	↪
↪	0.918703	98.665610	91.801889		0.094600		0.075602	↪
↪	85.705823							
10	3.166497	0.693846	1.314625	1.731167	0.271333	9.309159	0.747243	↪
↪	1.190337	98.831208	98.343473		0.093478		0.072439	↪
↪	91.895308							
11	2.825667	0.583369	0.904537	1.189014	0.262987	9.981676	0.681155	↪
↪	1.225510	98.784140	81.271593		0.082747		0.070825	↪
↪	79.849082							
12	2.809180	0.530423	1.155803	1.432078	0.286258	9.875173	0.671053	↪
↪	1.142482	97.475070	86.905404		0.091357		0.067388	↪
↪	83.232849							

13	3.948492	0.670550	0.752039	1.465636	0.298577	9.923262	0.650624	┘
↳	1.035515	99.709877	86.863968		0.078466		0.078685	┘
↳	84.678929							
14	3.166497	0.501950	1.314625	1.731167	0.271333	10.083053	0.864865	┘
↳	1.333853	98.472597	87.101933		0.098186		0.074785	┘
↳	80.411445							
15	3.639483	0.693846	0.795982	1.343432	0.251777	10.083053	0.512000	┘
↳	1.333853	98.936222	87.735506		0.081644		0.072328	┘
↳	86.406049							
16	3.166497	0.789794	1.314625	1.731167	0.271333	10.083053	0.512000	┘
↳	1.046821	98.459342	86.791318		0.095300		0.070400	┘
↳	86.569815							
17	3.639483	0.693846	0.795982	1.343432	0.251777	10.083053	0.629622	┘
↳	1.190337	98.863927	84.276903		0.081688		0.071920	┘
↳	82.559993							
18	3.166497	0.693846	1.314625	1.731167	0.251777	10.083053	0.512000	┘
↳	0.903305	99.341362	89.716301		0.092448		0.074566	┘
↳	87.197422							
19	4.111200	0.732915	1.266471	2.273687	0.304414	9.929749	0.433913	┘
↳	0.845490	97.362165	101.316388		0.095063		0.065988	┘
↳	96.759174							

	final_R50_oversize	P80_ens_mean	P80_ens_std	R95_ens_mean	R95_ens_std	
↳	final_feasible	ens_feasible	safe_feasible	design_score	boundary_penalty	┘
↳	robust_design_score					
0	38.441772	98.530183	0.276672	87.933757	3.950814	┘
↳	True	True	True	5.051461	0.000000	┘
↳	5.051461					
1	36.251774	98.527925	0.358066	83.469422	3.691210	┘
↳	True	True	True	4.890897	0.071983	┘
↳	5.106846					
2	32.081176	98.561214	0.620543	77.828987	3.509766	┘
↳	True	True	True	5.450756	0.000000	┘
↳	5.450756					
3	35.894790	98.456395	0.467913	91.252647	5.175062	┘
↳	True	True	True	5.461785	0.000000	┘
↳	5.461785					
4	32.180152	98.533873	0.661069	83.855754	5.006998	┘
↳	True	True	True	5.515397	0.000000	┘
↳	5.515397					
5	38.135962	98.528999	0.965068	81.851145	2.736207	┘
↳	True	True	True	5.626766	0.000000	┘
↳	5.626766					
6	34.042642	98.592481	0.641508	76.829547	1.873050	┘
↳	True	True	True	5.639042	0.000000	┘
↳	5.639042					

7	36.197564	98.562062	0.694935	85.022119	3.444496	␣
↩	True	True	True	5.672049	0.000000	␣
↩	5.672049					
8	35.388345	98.534630	0.500376	82.380356	1.958773	␣
↩	True	True	True	5.129068	0.181065	␣
↩	5.672262					
9	37.826083	98.551545	0.576920	88.184502	3.256693	␣
↩	True	True	True	5.692893	0.000000	␣
↩	5.692893					
10	42.550995	98.498801	0.317637	99.027301	16.280191	␣
↩	True	True	True	5.731649	0.000000	␣
↩	5.731649					
11	38.290429	98.539464	0.922342	84.318900	2.875070	␣
↩	True	True	True	5.752116	0.000000	␣
↩	5.752116					
12	37.148297	98.554827	0.671541	90.831867	3.730139	␣
↩	True	True	True	5.790242	0.000000	␣
↩	5.790242					
13	39.238340	98.418411	0.568394	85.956567	1.609623	␣
↩	True	True	True	5.797646	0.000000	␣
↩	5.797646					
14	36.203124	98.516060	0.724465	82.816194	5.423965	␣
↩	True	True	True	5.378081	0.181065	␣
↩	5.921276					
15	38.749995	98.495173	0.766151	86.194389	2.015127	␣
↩	True	True	True	5.444118	0.181065	␣
↩	5.987312					
16	38.192267	98.428310	0.491628	84.781001	2.788591	␣
↩	True	True	True	5.447048	0.181065	␣
↩	5.990242					
17	37.963994	98.471466	0.701138	84.189872	1.821352	␣
↩	True	True	True	5.448323	0.181065	␣
↩	5.991518					
18	39.306345	98.525740	0.715178	85.286242	3.597404	␣
↩	True	True	True	5.467329	0.181065	␣
↩	6.010523					
19	42.860981	98.494212	0.755044	96.835453	4.364247	␣
↩	True	True	True	6.049099	0.000000	␣
↩	6.049099					

1.9 9. Diagnostics and bounds check

```
[16]: selected_summary = selected_designs[input_cols + [
    "final_P80", "final_R95", "final_fines_frac", "final_oversize_frac",
    "final_R50_fines", "final_R50_oversize",
    "P80_ens_mean", "P80_ens_std", "R95_ens_mean", "R95_ens_std",
```

```

    "design_score", "boundary_penalty", "robust_design_score",
]]).describe().T

display(selected_summary)

bounds_rows = []
for col in input_cols:
    bounds_rows.append({
        "feature": col,
        "selected_min": selected_designs[col].min(),
        "selected_max": selected_designs[col].max(),
        "allowed_min": input_bounds[col]["min"],
        "allowed_max": input_bounds[col]["max"],
        "inside_bounds": (
            selected_designs[col].min() >= input_bounds[col]["min"]
            and selected_designs[col].max() <= input_bounds[col]["max"]
        ),
    })

bounds_check = pd.DataFrame(bounds_rows)
display(bounds_check)

```

		count	mean	std	min	25%	50%
↪ 75%	max						
energy	20.0	3.388203	0.432321	2.769840	3.117975	3.355683	
↪ 3.676497	4.111200						
angle_rad	20.0	0.675099	0.101456	0.501950	0.579074	0.693846	
↪ 0.742868	0.856038						
coupling	20.0	1.088042	0.279169	0.752039	0.820889	1.046774	
↪ 1.314625	1.620353						
strength	20.0	1.618387	0.374418	1.143469	1.343432	1.516952	
↪ 1.731167	2.416876						
porosity	20.0	0.277723	0.019303	0.251777	0.264408	0.271333	
↪ 0.293259	0.306724						
gravity	20.0	9.947327	0.171099	9.309159	9.914394	9.962109	
↪ 10.083053	10.083053						
atmosphere	20.0	0.660597	0.149585	0.422767	0.512000	0.676104	
↪ 0.802581	0.864865						
shape_factor	20.0	1.057618	0.187216	0.775107	0.890069	1.046821	
↪ 1.196302	1.333853						
final_P80	20.0	98.520519	0.624189	97.362165	98.122394	98.656059	
↪ 98.839388	99.709877						
final_R95	20.0	85.931067	6.397745	74.922776	81.553246	86.372019	
↪ 88.044955	101.316388						
final_fines_frac	20.0	0.088494	0.007092	0.077125	0.082482	0.089029	
↪ 0.094716	0.099837						

```

final_oversize_frac    20.0    0.071152  0.003953   0.062362   0.069794   0.071798
↳ 0.073040    0.078685
final_R50_fines        20.0  82.774360  5.859489  72.164253  79.583878  82.896421
↳ 86.332827   96.759174
final_R50_oversize     20.0  37.347251  2.772646  32.081176  36.121871  37.895038
↳ 38.518828   42.860981
P80_ens_mean           20.0  98.516089  0.045052  98.418411  98.494933  98.528462
↳ 98.542484   98.592481
P80_ens_std            20.0   0.619829  0.181137   0.276672   0.498189   0.651289
↳ 0.717499    0.965068
R95_ens_mean           20.0  85.942301  5.418824  76.829547  83.306115  84.901560
↳ 87.996443   99.027301
R95_ens_std            20.0   3.955439  3.113413   1.609623   2.555937   3.477131
↳ 4.054173   16.280191
design_score            20.0   5.521788  0.273642   4.890897   5.446315   5.491363
↳ 5.702582    6.049099
boundary_penalty       20.0   0.057919  0.084229   0.000000   0.000000   0.000000
↳ 0.181065    0.181065
robust_design_score    20.0   5.695544  0.278727   5.051461   5.598924   5.712271
↳ 5.937785    6.049099

      feature  selected_min  selected_max  allowed_min  allowed_max
↳inside_bounds
0      energy      2.769840      4.111200      0.500000      5.000000
↳True
1      angle_rad    0.501950      0.856038      0.261799      1.570796
↳True
2      coupling     0.752039      1.620353      0.200000      1.700000
↳True
3      strength     1.143469      2.416876      0.400000      4.200000
↳True
4      porosity     0.251777      0.306724      0.000000      0.330000
↳True
5      gravity      9.309159     10.083053      1.020000     10.470000
↳True
6      atmosphere    0.422767      0.864865      0.000000      1.000000
↳True
7      shape_factor  0.775107      1.333853      0.700000      1.500000
↳True

```

```

[17]: # Save a compact candidate report and the selected diagnostic report.
candidate_report_path = REPORTS_DIR / "final_inverse_design_candidate_pool.csv"
selected_report_path = REPORTS_DIR / "final_inverse_design_selected_diagnostics.
↳csv"

```

```

candidate_results.sort_values("robust_design_score").head(25_000).
    ↳to_csv(candidate_report_path, index=False)
selected_designs.to_csv(selected_report_path, index=False)

print("Saved candidate report to:", candidate_report_path)
print("Saved selected diagnostics to:", selected_report_path)

```

Saved candidate report to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports/final_inverse_design_candidate_pool.csv
 Saved selected diagnostics to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports/final_inverse_design_selected_diagnostics.csv

1.10 10. Create official inverse-design submission

The challenge requires exactly these columns:

submission_id, energy, angle_rad, coupling, strength, porosity, gravity, atmosphere, shape_factor
 with exactly 20 rows.

```

[18]: design_submission = selected_designs[input_cols].copy().reset_index(drop=True)
design_submission.insert(0, "submission_id", np.arange(len(design_submission)))

expected_design_columns = ["submission_id"] + input_cols
design_submission = design_submission[expected_design_columns]

# Official challenge filename.
official_design_path = SUBMISSIONS_DIR / "design_submission.csv"
# Descriptive backup filename.
descriptive_design_path = SUBMISSIONS_DIR / "design_submission_final_robust.csv"

design_submission.to_csv(official_design_path, index=False)
design_submission.to_csv(descriptive_design_path, index=False)

print("Saved official design submission to:", official_design_path)
print("Saved descriptive backup to:", descriptive_design_path)
print("Shape:", design_submission.shape)
display(design_submission)

```

Saved official design submission to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/design_submission.csv
 Saved descriptive backup to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions/design_submission_final_robust.csv
 Shape: (20, 9)

	submission_id	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	shape_factor
0	0	3.598016	0.797647	1.291533	2.024534	0.304489	9.851536	0.422767	1.303159

1	0.819009	1	3.219857	0.653336	0.937745	1.371766	0.268351	10.031512	␣
2	0.800587	2	3.883410	0.550208	0.754591	1.428060	0.294039	9.893283	␣
3	0.429999	3	2.769840	0.856038	0.933836	1.186456	0.266355	9.856101	␣
4	0.715197	4	3.491508	0.545414	1.620353	2.249113	0.264882	9.960773	␣
5	0.841676	5	2.818390	0.657906	0.841158	1.143469	0.291200	9.963445	␣
6	0.808564	6	3.944120	0.566188	0.829191	1.568268	0.293000	9.997341	␣
7	0.682885	7	2.972409	0.772728	1.329016	1.663816	0.306724	9.953774	␣
8	0.864865	8	3.639483	0.789794	0.795982	1.343432	0.251777	10.083053	␣
9	0.611923	9	3.787539	0.728334	1.498127	2.416876	0.292087	9.921431	␣
10	0.747243	10	3.166497	0.693846	1.314625	1.731167	0.271333	9.309159	␣
11	0.681155	11	2.825667	0.583369	0.904537	1.189014	0.262987	9.981676	␣
12	0.671053	12	2.809180	0.530423	1.155803	1.432078	0.286258	9.875173	␣
13	0.650624	13	3.948492	0.670550	0.752039	1.465636	0.298577	9.923262	␣
14	0.864865	14	3.166497	0.501950	1.314625	1.731167	0.271333	10.083053	␣
15	0.512000	15	3.639483	0.693846	0.795982	1.343432	0.251777	10.083053	␣
16	0.512000	16	3.166497	0.789794	1.314625	1.731167	0.271333	10.083053	␣
17	0.629622	17	3.639483	0.693846	0.795982	1.343432	0.251777	10.083053	␣
18	0.512000	18	3.166497	0.693846	1.314625	1.731167	0.251777	10.083053	␣
19	0.433913	19	4.111200	0.732915	1.266471	2.273687	0.304414	9.929749	␣

1.11 11. Final format validation

```
[19]: assert design_submission.shape == (20, 9), "Design submission must have 20 rows
      and 9 columns."
      assert design_submission.columns.tolist() == expected_design_columns, "Design
      submission columns are incorrect."
```

```

assert design_submission["submission_id"].tolist() == list(range(20)),
    ↪ "submission_id must be 0 to 19."
assert np.isfinite(design_submission[input_cols].values).all(), "Design
    ↪ submission contains non-finite values."

for col in input_cols:
    assert (design_submission[col] >= input_bounds[col]["min"]).all(), f"{col}
    ↪ below allowed min."
    assert (design_submission[col] <= input_bounds[col]["max"]).all(), f"{col}
    ↪ above allowed max."

print("Design submission format validation passed.")

```

Design submission format validation passed.

1.12 12. Final notes

Use these two official files for submission:

outputs/submissions/prediction_submission.csv

outputs/submissions/design_submission.csv

This notebook is the final inverse-design source of truth. Earlier inverse-design notebooks remain useful as experiments.